# Natural Language Processing

**Prof. Ahmed Guessoum**
**The National Higher School of AI**

---

## Chapter 9

# The Transformer

# A very approximate timeline

- 1990 Static Word Embeddings
- 2003 Neural Language Model
- 2008 Multi-Task Learning
- 2015 Attention
- 2017 Transformer
- 2018 Contextual Word Embeddings and Pretraining
- 2019 Prompting

# The Transformer

- It is the standard architecture for building large language models.
- Transformer-based large language models have completely changed the field of speech and language processing.
- We will focus for now on left-to-right (sometimes called *causal* or *autoregressive*) *language modeling*.
- The **transformer** is a neural network with a specific structure that includes a mechanism called **self-attention** or multi-head attention.
- **Attention** : a way to build contextual representations of a token's meaning by *attending to* and integrating information from surrounding tokens, helping the model learn how tokens relate to each other over large spans.

# Attention

# Landmark paper!

## Attention Is All You Need


**Ashish Vaswani*** Google Brain avaswani@google.com

**Noam Shazeer*** Google Brain noam@google.com

**Niki Parmar*** Google Research nikip@google.com

**Jakob Uszkoreit*** Google Research usz@google.com

**Llion Jones*** Google Research llion@google.com

**Aidan N. Gomez*** † University of Toronto aidan@cs.toronto.edu

**Łukasz Kaiser*** Google Brain lukaszkaiser@google.com

**Illia Polosukhin*** ‡ illia.polosukhin@gmail.com

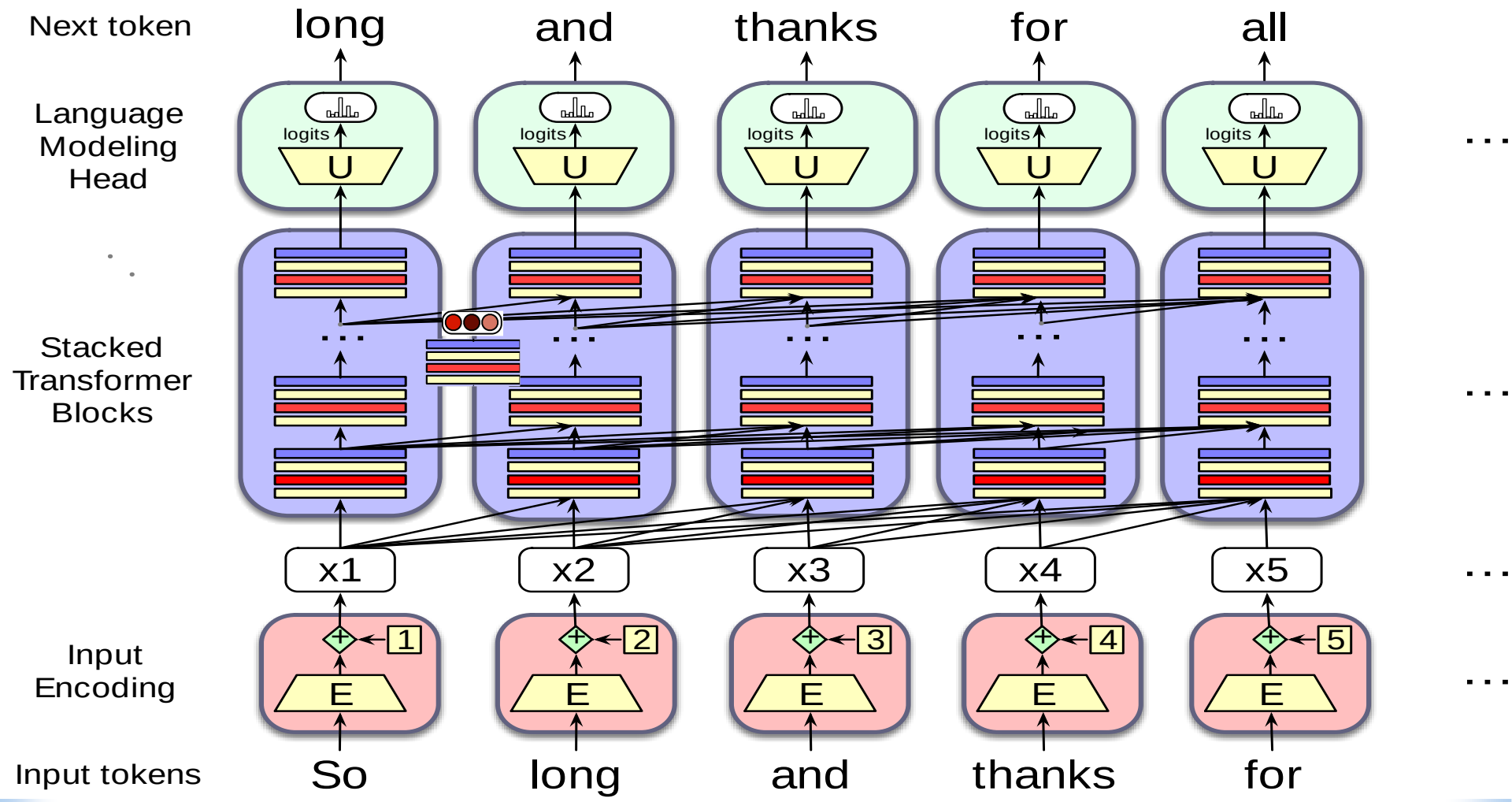

**Attention** : a way to build contextual representations of a token's meaning by *attending to* and integrating information from surrounding tokens, helping the model learn how tokens relate to each other over large spans.

# Landmark paper…

- **Groundbreaking**: The introduction of the Transformer was a breakthrough, as it moved away from earlier approaches like RNNs and LSTMs, offering superior scalability and performance.
- **Pioneering**: The Transformer paper pioneered the use of self-attention and showed how it could replace recurrent structures, which had been dominant for sequential data tasks.
- **Foundational**: The Transformer model is the cornerstone of many of the most powerful language models today, making the paper foundational to modern NLP research.
- **Transformative**: It transformed the landscape of NLP and deep learning, inspiring many subsequent innovations and applications.
- **Seminal**: "Attention is All You Need" is often considered a seminal work, meaning it has had a lasting and profound influence on the field.

# Instead of starting with the big picture

Let's consider the embeddings for an individual word from a particular layer



# The Transformer

- A transformer has 3 major components:
  - At the center are columns of transformer blocks.
  - Each block is a multilayer network
    - a multi-head attention layer,
    - feedforward network and
    - layer normalization

  that maps an input vector $x_i$ in column i (corresponding to input token i) to an output vector $h_i$.
  - The set of n blocks maps an entire context window of input vectors ($x_1$, …, $x_n$) to a window of output vectors ($h_1$, …, $h_n$) of the same length.
  - A column might contain from 12 to 96 or more stacked blocks.

# The Transformer

- The column of blocks is preceded by the input encoding component, which processes an input token into a contextual vector representation, using
  - an embedding matrix E and
  - a mechanism for encoding token position.
- Each column is followed by a language modeling head, which
  - takes the embedding output by the final transformer block,
  - passes it through an unembedding matrix U and
  - a softmax over the vocabulary

  to generate a single token for that column.

# Problem with static embeddings (word2vec)

- They are static! I.e. the embedding for a word doesn't reflect how its meaning changes in context.

`The chicken didn't cross the road because` **`it`** `was too tired`

- E.g. word *chicken*: always represented by the same fixed vector.
- A static vector for the word *it* might somehow encode that this is a pronoun used for animals and inanimate entities.

# Contextual Embedding

- In context *it* has a much richer meaning. E.g.:
  1. The chicken didn't cross the road because it was too tired.
  2. The chicken didn't cross the road because it was too wide.

# Contextual Embedding

- In context *it* has a much richer meaning. E.g.:
  1. The **chicken** didn't cross the road because it was too tired.
  2. The chicken didn't cross the **road** because it was too wide.
- So if we are to compute the meaning of this sentence:
  - it the meaning of it should be associated with ***chicken*** in the first sentence and
  - associated with the ***road*** in the second one,

➔ **sensitive to the context.**

# Contextual Embeddings

- Intuition: a representation of meaning of a word should be different in different contexts!
- **Contextual Embedding**: each word has a different vector that expresses different meanings depending on the surrounding words
- How to compute contextual embeddings?
  - **Attention**

# Contextual Embeddings

- Given the possible sentences:

1. `The chicken didn't cross the road because it was too` **`tired`**
2. `The chicken didn't cross the road because it was too` **`wide`**

- and the input:

`The chicken didn't cross the road because it`

- At this point we don't yet know which thing it is going to end up referring to! *chicken* or *road*?

- Other examples:

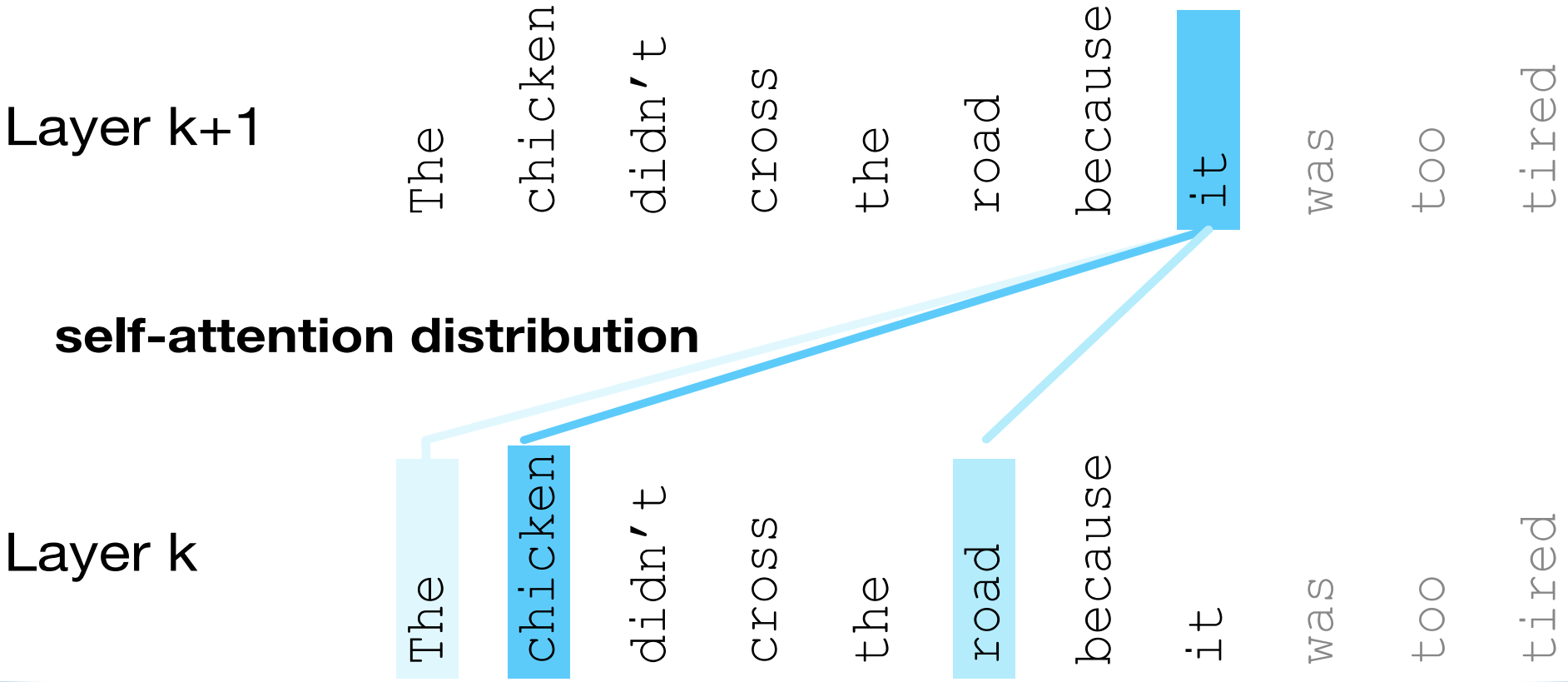
3. **`keys`** `to the cabinet` **`are`** `on the table.` (syntactic agreement)
4. `I walked along the` **`pond`**`, and noticed one of the trees along the` **`bank`**`.` (word sense disambiguation)

# Intuition of Attention

- These examples showcase that the contextual words that help us compute the meaning of words in context can be quite far away in the sentence or paragraph.
- **Transformers** can build contextual representations of word meaning, *contextual embeddings*, by integrating the meaning of these helpful contextual words.
- In a transformer, layer by layer, we build up richer and richer contextualized representations of the meanings of input tokens.
- At each layer, we compute the representation of a token *i* by combining information about *i* from the previous layer with information about the neighboring tokens to produce a contextualized representation for each word at each position.
- **Intuition of Attention:** Build up the contextual embedding from a word by selectively integrating information from all the neighboring words.
- We say that a word "*attends to*" some neighboring words more than others.

# Intuition of attention

columns corresponding to input tokens

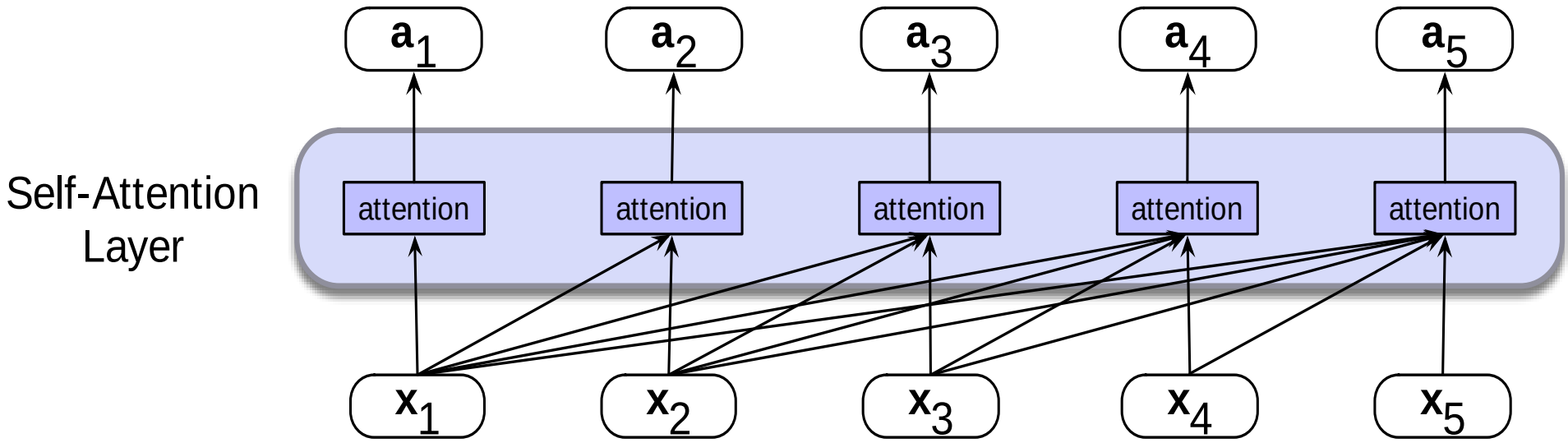


# Attention definition

- A mechanism for helping compute the embedding for a token by selectively **attending to** and **integrating** information from surrounding tokens (at the previous layer).

- More formally: a method for doing a weighted sum of vectors.

# Attention is left-to-right



# Simplified version of attention

Given a sequence of token embeddings:

$\mathbf{x}_1 \quad \mathbf{x}_2 \quad \mathbf{x}_3 \quad \mathbf{x}_4 \quad \mathbf{x}_5 \quad \mathbf{x}_6 \quad \mathbf{x}_7 \quad \mathbf{x}_i$

Produce: $\mathbf{a}_i$ a weighted sum of $\mathbf{x}_1$ through $\mathbf{x}_7$ (and $\mathbf{x}_i$) weighted by their similarity to $\mathbf{x}_i$

$$score(x_i, x_j) = x_i . x_j$$

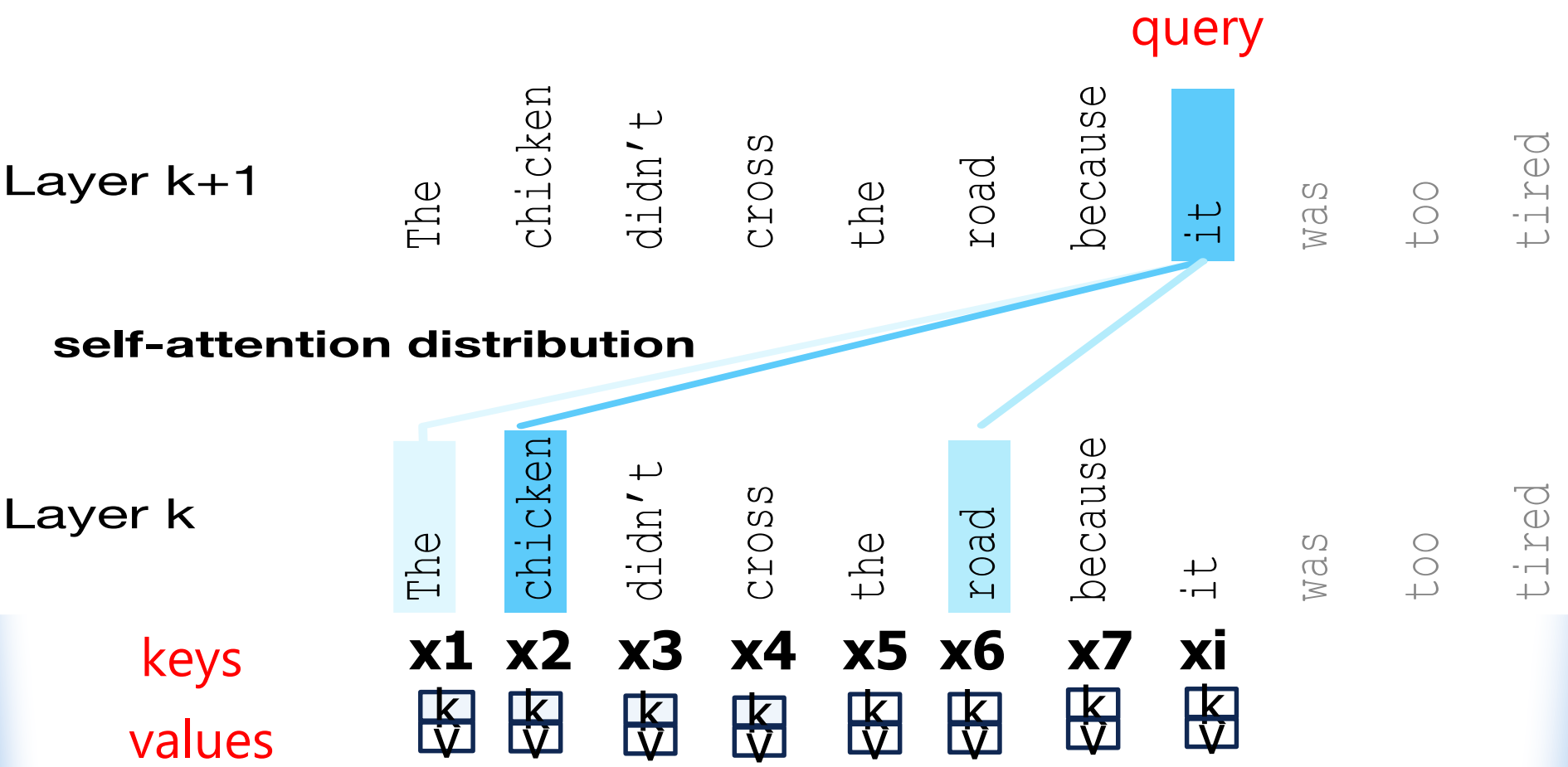
$$\alpha_{ij} = softmax\big(score(x_i, x_j)\big) \qquad \forall j \leq i$$

$$a_i = \sum_{j \leq i} \alpha_{ij} x_j$$

# An Actual Attention Head: slightly more complicated

- **High-level idea**: instead of using vectors (like $x_i$ and $x_4$) directly, we'll represent 3 separate roles each vector $\mathbf{x}_i$ plays:
  - **query**: as *the current element* being compared to the preceding inputs;
  - **key**: as *a preceding input* that is being compared to the current element to determine a similarity;
  - **value**: a value of a preceding element that gets weighted and summed.

# Intuition of attention



# Attention Head: slightly more complicated

- We'll use matrices to project each vector $\mathbf{x}_i$ into a representation of its role as **query**, **key**, or **value** :

  **query**: $\mathbf{W}^{\mathbf{Q}}$

  **key**: $\mathbf{W}^{\mathbf{K}}$

  **value**: $\mathbf{W}^{\mathbf{V}}$

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^{\mathbf{Q}}; \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^{\mathbf{K}}; \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^{\mathbf{V}}$$

# Attention Head: slightly more complicated

- Given these 3 representation of $\mathbf{x}_i$

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K; \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

- To compute the similarity of current element $\mathbf{x}_i$ with some prior element $\mathbf{x}_j$, we'll use dot product between $\mathbf{q}_i$ and $\mathbf{k}_j$.
- And instead of summing up $\mathbf{x}_j$ , we'll sum up $\mathbf{v}_j$

# Final equations for one attention head

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^{\mathbf{Q}}; \quad \mathbf{k}_j = \mathbf{x}_j \mathbf{W}^{\mathbf{K}}; \quad \mathbf{v}_j = \mathbf{x}_j \mathbf{W}^{\mathbf{V}}$$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i$$

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{v}_j$$

# Calculating the value of a3

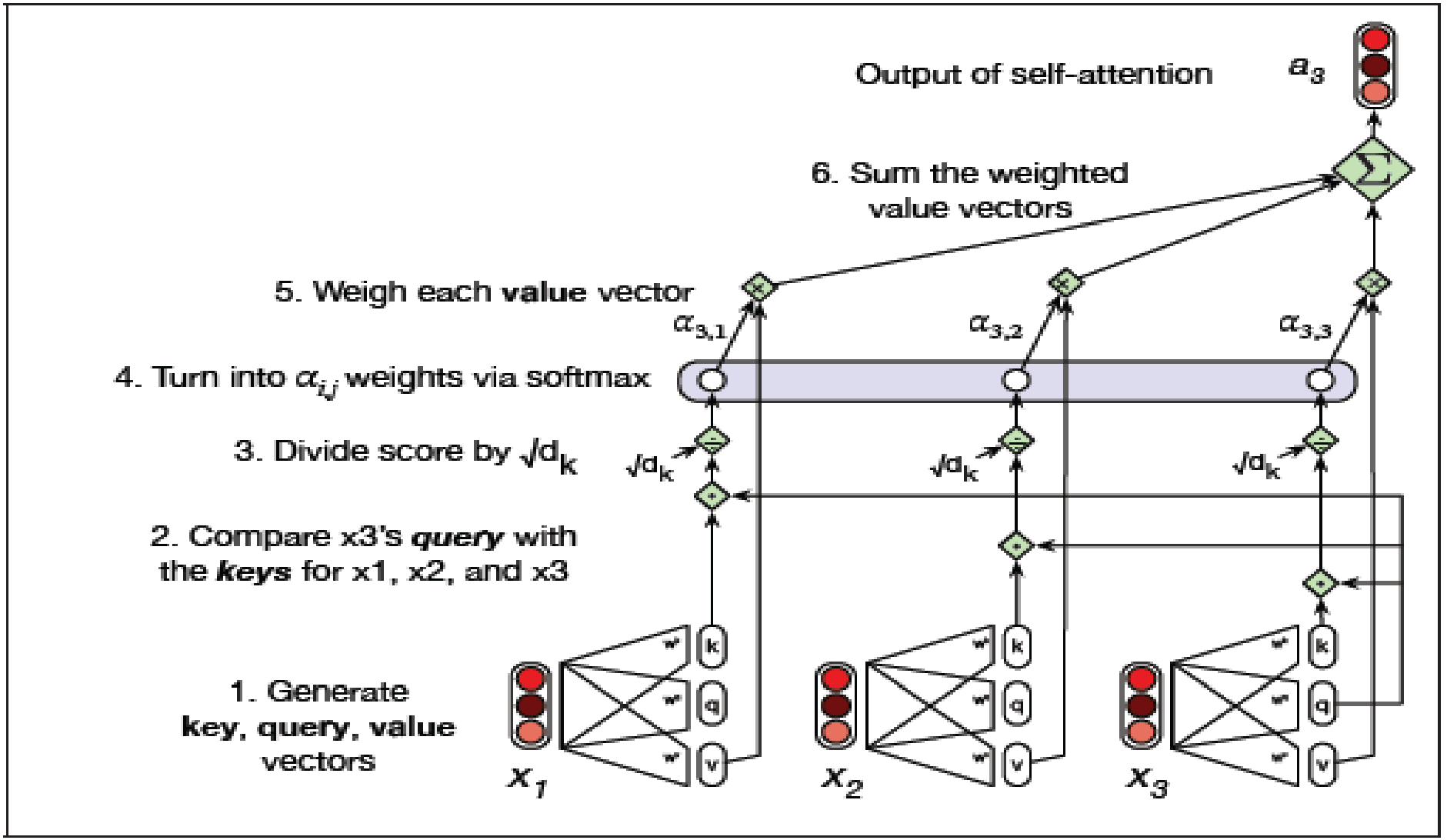



**Figure 9.4** Calculating the value of $a_3$, the third element of a sequence using causal (left-to-right) self-attention.

# Learning the weight matrices $W^Q$, $W^K$, $W^V$

- $W^Q$, $W^K$, and $W^V$ are learned **during the training process** via backpropagation, just like any other learnable parameter in a neural network.
- **Preliminary key points:**
  - **Transformers** (and their attention mechanisms) are trained using **supervised learning**
  - A loss function (such as cross-entropy loss) is optimised by adjusting the parameters of the network (including $W^Q$, $W^K$, and $W^V$).
  - The goal of the attention mechanism is to compute a *weighted sum of the values*, where the weights are determined by the attention scores (computed from the *queries* and *keys*).
  - **Backpropagation** is used to adjust the weights of $W^Q$, $W^K$, and $W^V$, based on the gradients computed from the loss function.

# Learning the weight matrices $W^Q$, $W^K$, $W^V$

1. **The Setup:**
   - At each position in a sequence, we have an embedding vector for each token (e.g., $\mathbf{e_i}$ for token i).
   - The attention mechanism transforms these embeddings into **Query**, **Key**, and **Value** vectors for each token, using the weight matrices $W^Q$, $W^K$, and $W^V$.
2. **How the Attention Mechanism Works**: The input embeddings for a sequence (say, $\mathbf{e_1,e_2,...,e_n}$) are transformed into Query, Key, and Value vectors as follows:
   - **Query (Q)**: $Q_i = W^Q \cdot e_i$

   where $Q_i$ is the query vector for token *i*, and $W^Q$ is a learnable weight matrix that transforms the embedding $e_i$ into the *Query* vector.

# Learning the weight matrices $W^Q$, $W^K$, $W^V$

- **Key (K)**: $K_i = W^K \cdot e_i$
  where $K_i$ is the key vector for token *i*, and $W^K$ is a learnable weight matrix that transforms the embedding $e_i$ into the *Key* vector.
- **Value (V)**: $V_i = W^V \cdot e_i$
  where Vi is the value vector for token i, and $W^V$ is a learnable weight matrix that transforms the embedding ei into the Value vector.

# Learning the weight matrices $W^Q$, $W^K$, $W^V$

3. **Attention Calculation**: For each **Query** $Q_i$ , we compute the **attention scores** by measuring the similarity between $Q_i$ and the **Keys** of all tokens (including its own).
   - This is usually done using the dot product (often scaled by the square root of the dimension of the key vectors):

$$\textbf{Attention Score}(i, j) = \frac{Q_i \cdot K_j}{\sqrt{d_k}}$$

where $d_k$ is the dimension of the *Key* vector.

- These scores are then passed through a **softmax function** giving:

$$\textbf{Attention Weights}(i, j) = softmax(\frac{Q_i \cdot K_j}{\sqrt{d_k}})$$

- Finally, the *Value vectors* are weighted by these attention scores to compute the output for the Query $Q_i$:

$$\textbf{Attention Weights}(i) = \sum_j \textbf{Attention Weights}(i, j) \cdot V_j$$

## 4. Training and Backpropagation:

- The goal during training is to minimize a **loss function** (such as cross-entropy loss in the case of language modeling or classification).
- Backpropagation is used to compute the **gradients** of the loss with respect to the model parameters (including $W^Q$, $W^K$, and $W^V$).
- **Gradients**: The gradients indicate how to adjust $W^Q$, $W^K$, and $W^V$ to reduce the loss.

- For example, consider the loss with respect to the Query weight matrix $W^Q$: $\frac{\partial L}{\partial W^Q}$
- This gradient tells the model how to adjust $W^Q$ based on the error between the predicted output and the target.
  - The gradients are then used to update the weights using an optimization algorithm (typically **stochastic gradient descent** or one of its variants, such as **Adam**). The updates are as : $W^Q = W^Q - \eta \frac{\partial L}{\partial W^Q}$ (η: learning rate)
- The same process happens for $W^K$ and $W^V$, with their respective gradients being computed and used to update them.

# Actual Attention: slightly more complicated

- Instead of one attention head, we'll have lots of them *(h* of them*)!*
- Intuition: each head might be attending to the context for different purposes
  - Different linguistic relationships or patterns in the context

$$\mathbf{q}_i^c = \mathbf{x}_i\mathbf{W}^{\mathbf{Qc}};\quad \mathbf{k}_j^c = \mathbf{x}_j\mathbf{W}^{\mathbf{Kc}};\quad \mathbf{v}_j^c = \mathbf{x}_j\mathbf{W}^{\mathbf{Vc}};\quad \forall\, c \quad 1 \le c \le h$$

$$\text{score}^c(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i^c \cdot \mathbf{k}_j^c}{\sqrt{d_k}}$$

$$\alpha_{ij}^c = \text{softmax}(\text{score}^c(\mathbf{x}_i, \mathbf{x}_j)) \;\; \forall j \le i$$

$$\mathbf{head}_i^c = \sum_{j \le i} \alpha_{ij}^c \mathbf{v}_j^c$$

$$\mathbf{a}_i = (\mathbf{head}^1 \oplus \mathbf{head}^2 \ldots \oplus \mathbf{head}^h)\mathbf{W}^O$$

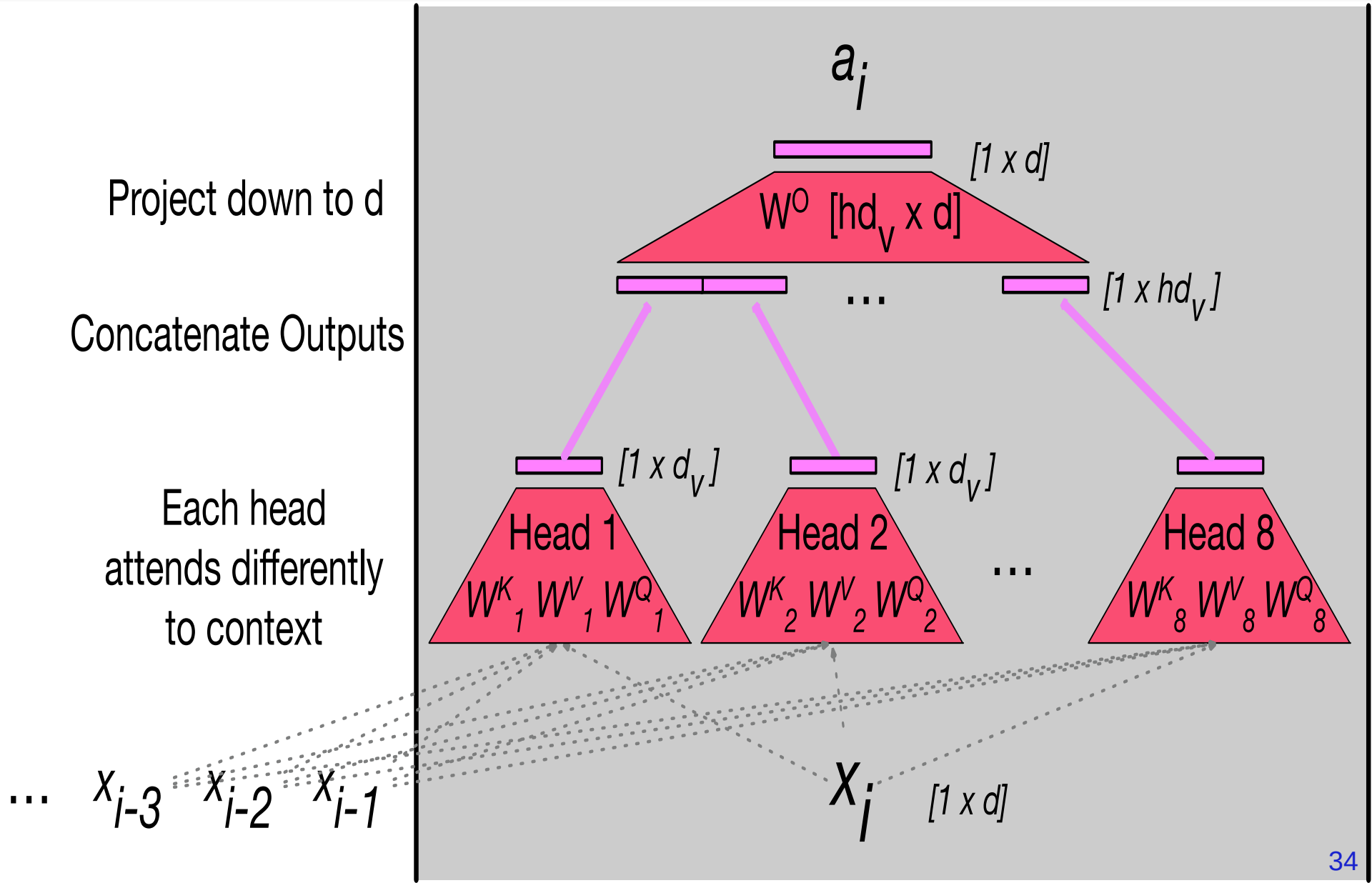
$$\text{MultiHeadAttention}(\mathbf{x}_i, [\mathbf{x}_1, \cdots, \mathbf{x}_N]) = \mathbf{a}_i$$

# Why multi-head attention?

**Different purposes**: Each attention head might learn to attend to different types of contextual relationships or dependencies between tokens. E.g.:

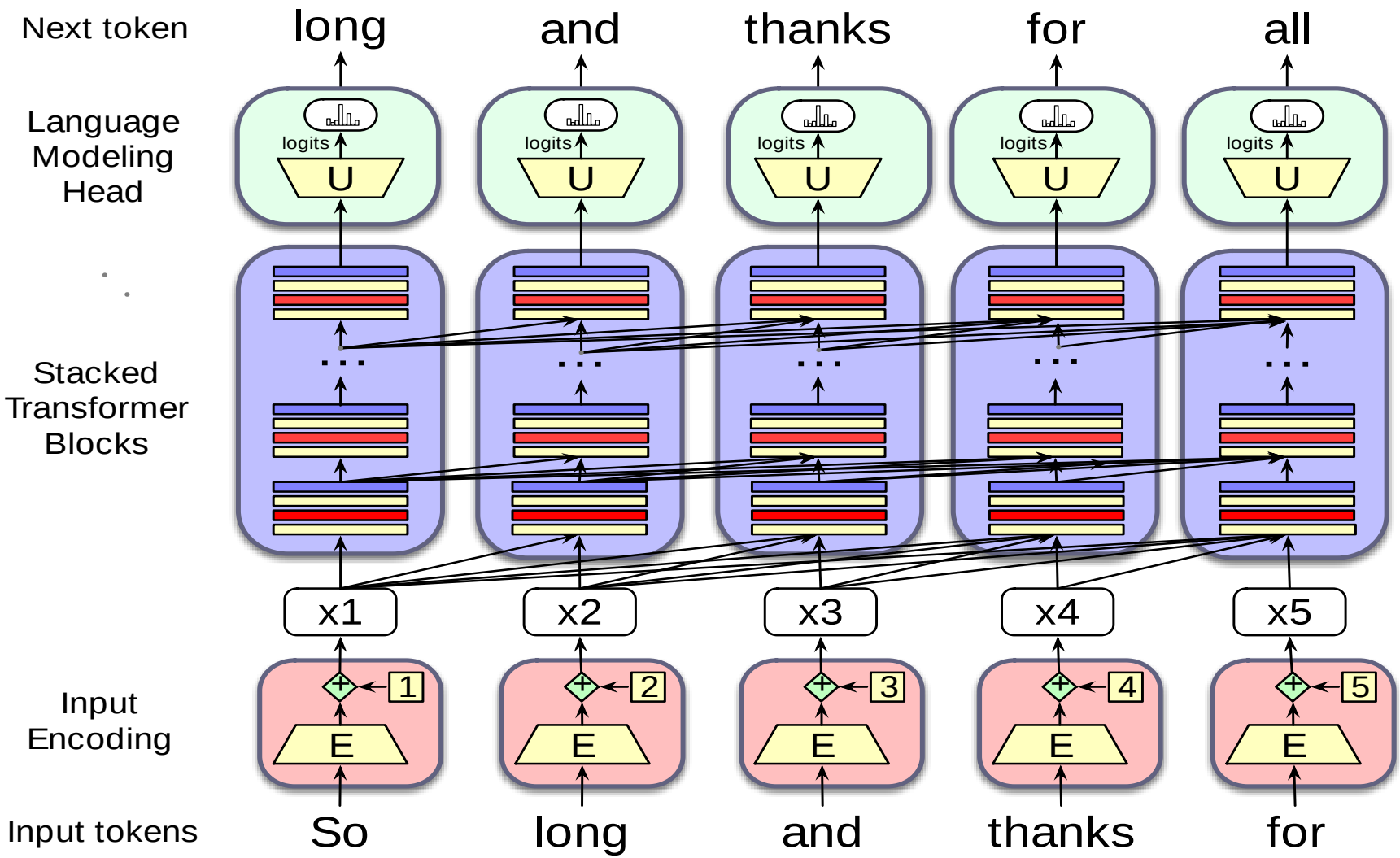
- One attention head might focus on **local** dependencies, paying attention to nearby words or tokens.
- Another head might focus on **long-range dependencies**, capturing relationships between tokens that are far apart in the sequence.
- A third head could focus on **syntax** or **semantics**, attending to grammatical or meaning-related patterns in the text.
- Another head might capture **word alignment** across different phrases or languages in tasks like translation.

# Multi-head attention



# The Transformer Block

# Reminder: transformer language model

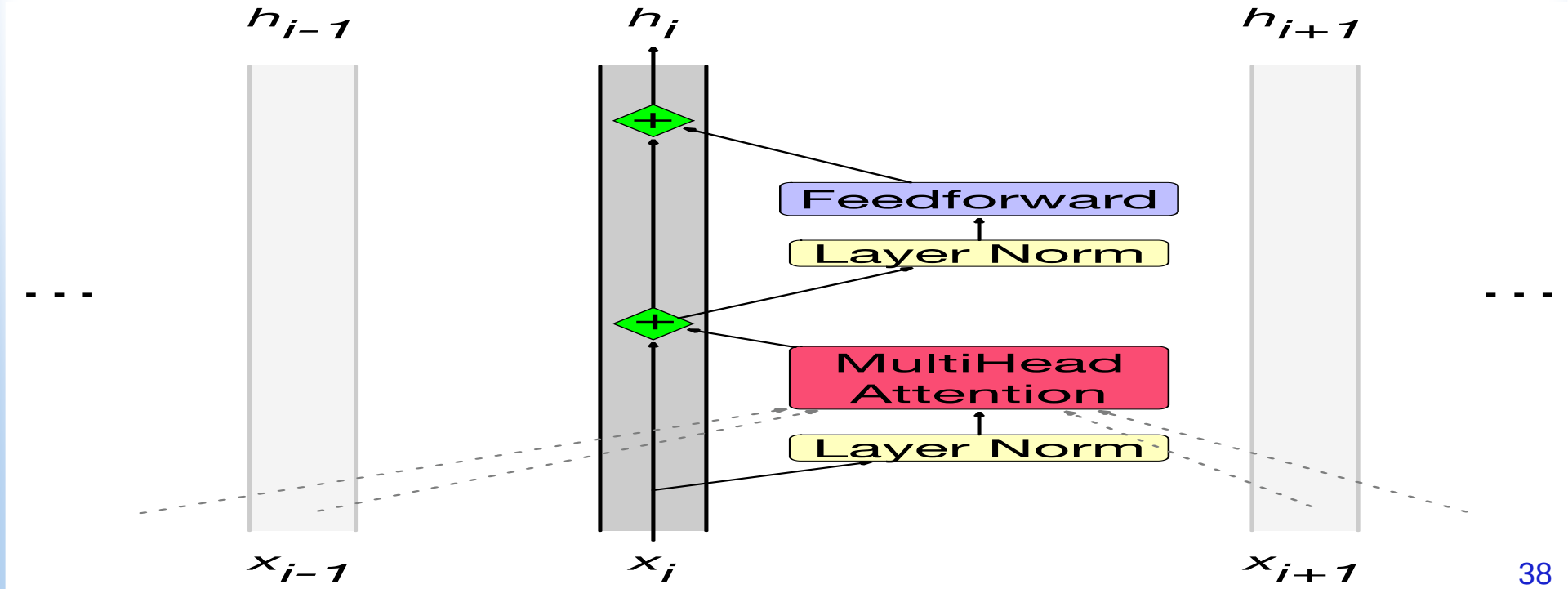


# Inside the transformer Block

- The self-attention calculation lies at the core of the transformer block
- Recall that the transformer block includes:
  - the self-attention layer,
  - a feedforward layer,
  - residual connections (not mentioned earlier), and
  - normalizing layers (“layer norm”).

# The residual stream

- Here, the processing of an individual token *i* through the transformer block is considered as a single stream of d-dimensional representations for token position *i*.
- This residual stream starts with the original input vector, and the various components read their input from the residual stream and add their output back into the stream.



# The residual stream

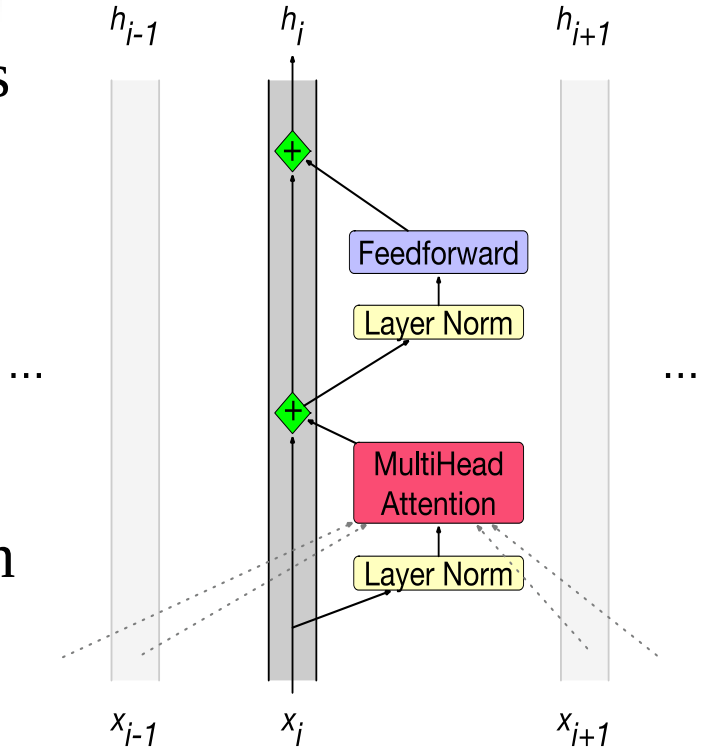
- The *initial vector* for token *i* is passed through a *layer norm* and *attention layer*, and
- The result is added back into the stream, in this case to the original input vector $x_i$.
- This *summed vector* is again passed through another *layer norm* and a *feedforward layer*,
- The output of those is added back into the residual,
- This result, referred to as $h_i$, is the output of the transformer block for *token i*.
- In earlier descriptions the **residual stream** was often described as residual connections.

# Feedforward Layer

- An FFN layer is needed for nonlinearities

$$\mathrm{FFN}(\mathbf{x}_i) = \mathrm{ReLU}(\mathbf{x}_i\mathbf{W}_1 + b_1)\mathbf{W}_2 + b_2$$

- A fully-connected 2-layer network (1hidden layer).
- Weights: the same for each token position $i$, but different from layer to layer.
- Dimensionality $d_{ff}$ of the hidden layer of the FFN commonly made larger than the model dimensionality d.
- (For example, in the original transformer model, d = 512 and $d_{ff}$ = 2048.)



# Layer Norm

- Vector $\mathbf{x}_i$ is normalized twice in the transformer block.
- Layer norm applies a normalization operation to a single vector in a hidden layer.
- The normalised vector $\hat{x}$ has a mean of 0 and standard deviation of 1.

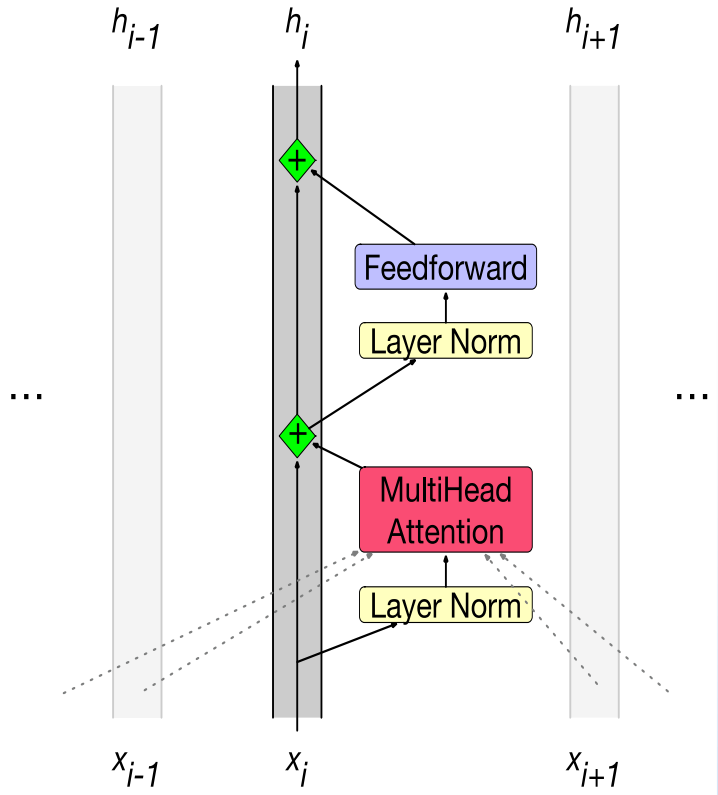
$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i$$

$$\sigma = \sqrt{\frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2}$$

$$\hat{\mathbf{x}} = \frac{(\mathbf{x} - \mu)}{\sigma}$$

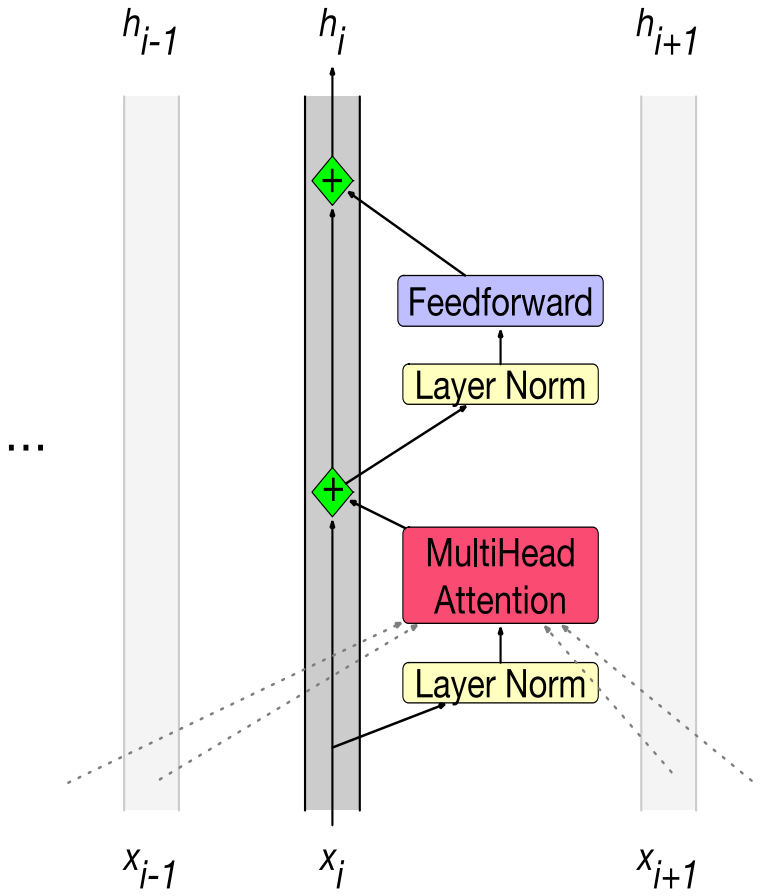
$$\text{LayerNorm}(\mathbf{x}) = \gamma\frac{(\mathbf{x} - \mu)}{\sigma} + \beta$$

Where $\gamma$ and $\beta$ are 2 learnable parameter vectors



# Putting together a single transformer block

- Function computed by a **transformer block**: broken down with one equation for each component computation, using **t** (of shape [1 x d]) to stand for transformer.




$$\mathbf{t}_i^1 = \text{LayerNorm}(\mathbf{x}_i)$$

$$\mathbf{t}_i^2 = \text{MultiHeadAttention}(\mathbf{t}_i^1, [\mathbf{x}_1^1, \cdots, \mathbf{x}_N^1])$$

$$\mathbf{t}_i^3 = \mathbf{t}_i^2 + \mathbf{x}_i$$

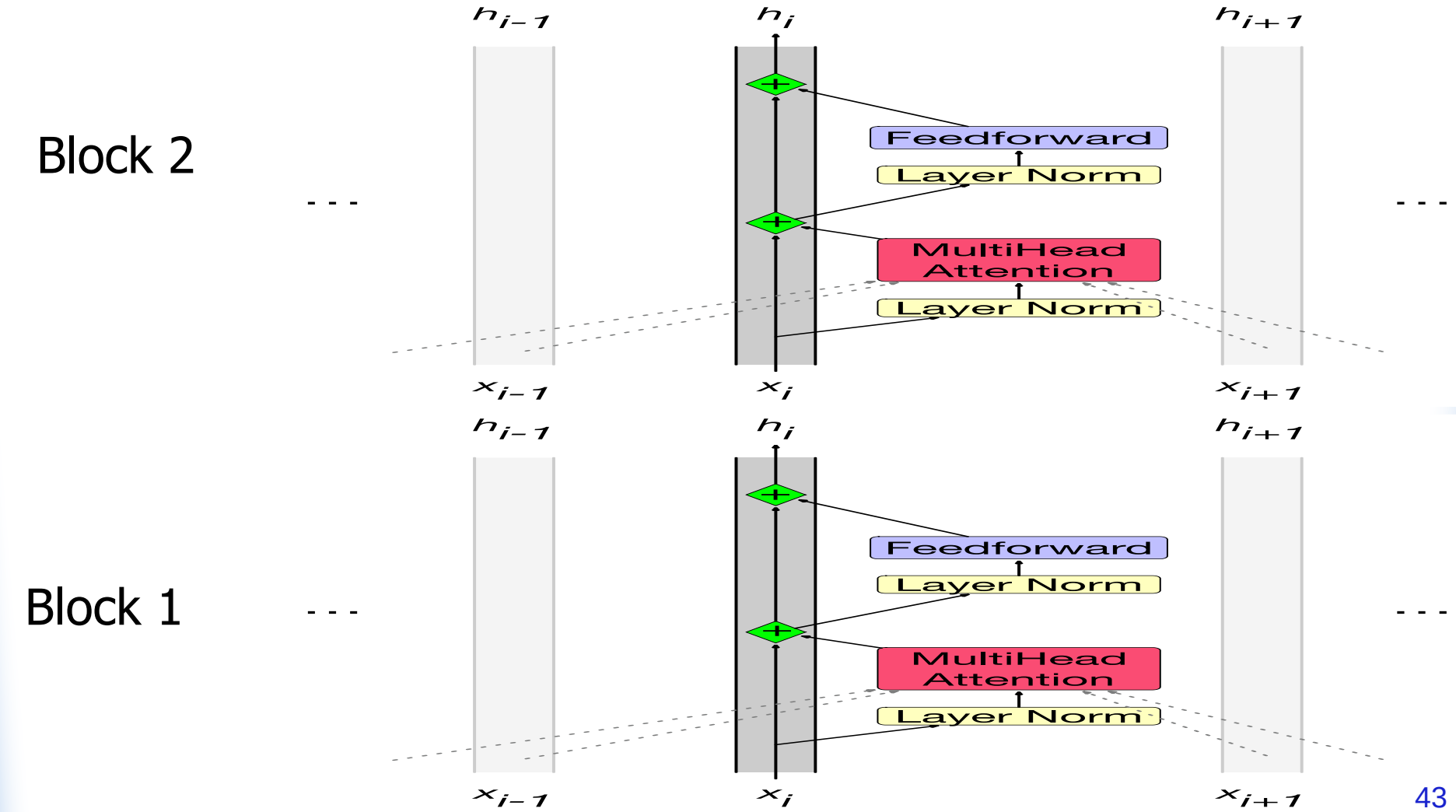
$$\mathbf{t}_i^4 = \text{LayerNorm}(\mathbf{t}_i^3)$$

$$\mathbf{t}_i^5 = \text{FFN}(\mathbf{t}_i^4)$$

$$\mathbf{h}_i = \mathbf{t}_i^5 + \mathbf{t}_i^3$$

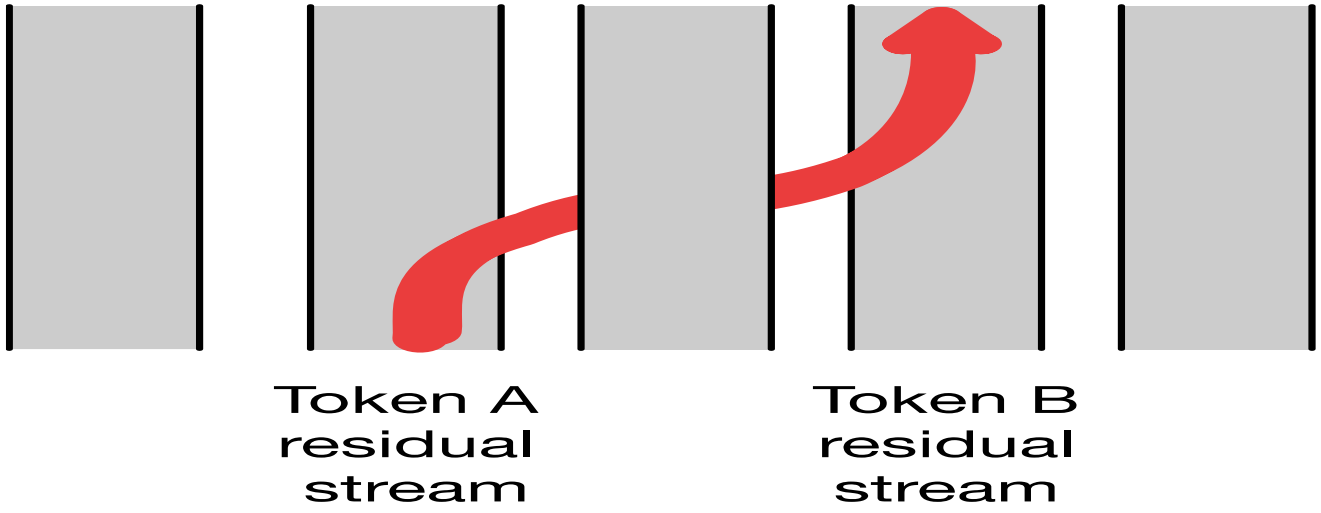
# A transformer as a stack of blocks

- A **Transformer** is a stack of these blocks;
- All the vectors are of the same dimensionality d.



# Residual Streams & Attention

- Notice that all parts of the transformer block apply to 1 residual stream (1 token),
- Except attention, which takes information from other tokens.
- Elhage et al. (2021) show that we can view attention heads as literally moving information from the residual stream of a neighboring token into the current stream.



# Parallelizing Attention Computation

# Parrallelizing the Computation

- Recall that the attention computation performed for each token to compute $\mathbf{a}_i$ is independent of the computation for each other token.
- This is also true for all the computation in the transformer block, computing $h_i$ from the input $x_i$.
- As such we can easily parallelize the entire computation, taking advantage of efficient matrix multiplication routines.

# Parallelizing computation for a single head

- Indeed, for attention/transformer block we've been computing a **single** output at a **single** time step *i* in a **single** residual stream.
- But, for a single head, we can pack the $N$ tokens of the input sequence into a single matrix $\mathbf{X}$ of size $[N \times d]$.

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^{\mathbf{Q}};\ \mathbf{K} = \mathbf{X}\mathbf{W}^{\mathbf{K}};\ \mathbf{V} = \mathbf{X}\mathbf{W}^{\mathbf{V}}$$

- Each row of $\mathbf{X}$ is the embedding of one token of the input.
- $\mathbf{X}$ can have 1K-32K rows, each of the dimensionality of the embedding $d$ (the **model dimension**)

# $QK^T$

- Now we can do a single matrix multiply to combine Q and $K^T$ . (Result of shape [N x N]

N

| | | | |
|---|---|---|---|
| q1·k1 | q1·k2 | q1·k3 | q1·k4 |
| q2·k1 | q2·k2 | q2·k3 | q2·k4 |
| q3·k1 | q3·k2 | q3·k3 | q3·k4 |
| q4·k1 | q4·k2 | q4·k3 | q4·k4 |

N

# Parallelizing attention

- Scale the scores, take the softmax, and then multiply the result by V resulting in a matrix of shape $N \times d$
  - An attention vector for each input token

$$\mathbf{A} = \text{softmax}\left(\text{mask}\left(\frac{\mathbf{QK}^\intercal}{\sqrt{d_k}}\right)\right)\mathbf{V}$$

- The entire self-attention step for an entire sequence of N tokens for one head has been reduced to the above computation.

# Masking out the future

$$\mathbf{A} = \text{softmax}\left(\text{mask}\left(\frac{\mathbf{Q}\mathbf{K}^\intercal}{\sqrt{d_k}}\right)\right)\mathbf{V}$$

- What is this *mask* function?
- $QK^T$ has a score for each query dot every key, *including those that follow the query*.
- Guessing the next word is pretty simple if you already know it!

# Masking out the future

$$\mathbf{A} = \text{softmax}\left(\text{mask}\left(\frac{\mathbf{Q}\mathbf{K}^\intercal}{\sqrt{d_k}}\right)\right)\mathbf{V}$$

- Add $-\infty$ to cells in the upper triangle
- The softmax will turn it to 0

| | | | |
|---|---|---|---|
| q1·k1 | −∞ | −∞ | −∞ |
| q2·k1 | q2·k2 | −∞ | −∞ |
| q3·k1 | q3·k2 | q3·k3 | −∞ |
| q4·k1 | q4·k2 | q4·k3 | q4·k4 |

N

N

# Attention is quadratic

$$\mathbf{A} = \text{softmax}\left(\text{mask}\left(\frac{\mathbf{Q}\mathbf{K}^\intercal}{\sqrt{d_k}}\right)\right)\mathbf{V}$$

- Attention is quadratic in the length of the input.

➔expensive to compute attention over very long documents (like entire novels).

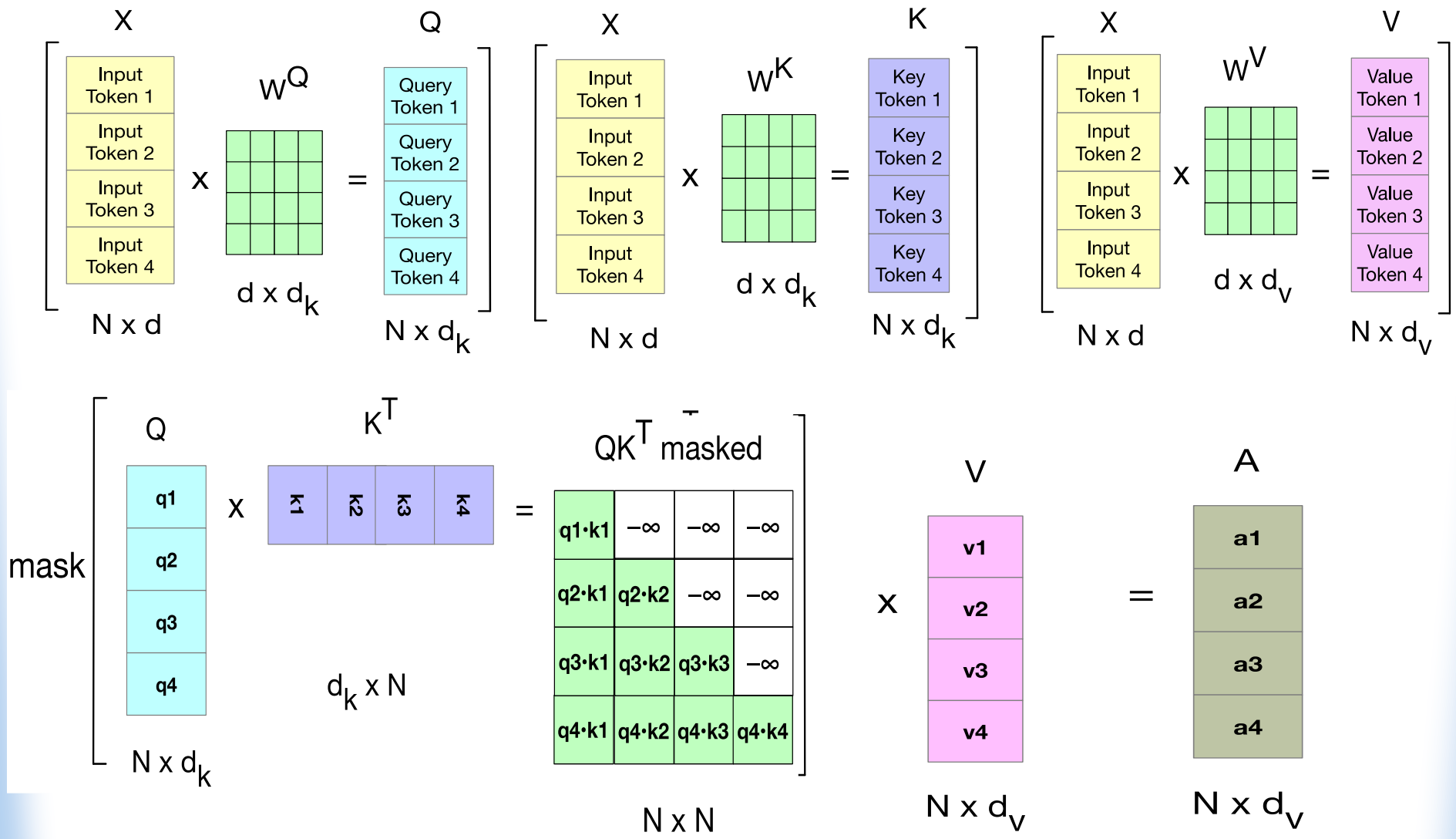
- Nonetheless modern LLMs manage to use quite long contexts of thousands or tens of thousands of tokens.

N

| | | | |
|---|---|---|---|
| q1·k1 | −∞ | −∞ | −∞ |
| q2·k1 | q2·k2 | −∞ | −∞ |
| q3·k1 | q3·k2 | q3·k3 | −∞ |
| q4·k1 | q4·k2 | q4·k3 | q4·k4 |

N

# Schematic of the attention computation for a single attention head in parallel



# Parallelizing Multi-head Attention

- Thus for each head *i*

$$\mathbf{Q}^i = \mathbf{X}\mathbf{W}^{Qi};\ \ \mathbf{K}^i = \mathbf{X}\mathbf{W}^{Ki};\ \ \mathbf{V}^i = \mathbf{X}\mathbf{W}^{Vi}$$

$$\mathbf{head}_i = \text{SelfAttention}(\mathbf{Q}^i, \mathbf{K}^i, \mathbf{V}^i) = \text{softmax}\left(\frac{\mathbf{Q}^i\mathbf{K}^{iT}}{\sqrt{d_k}}\right)\mathbf{V}^i$$

- And for all the heads:

$$\text{MultiHeadAttention}(\mathbf{X}) = (\mathbf{head}_1 \oplus \mathbf{head}_2 \ldots \oplus \mathbf{head}_h)\mathbf{W}^O$$

# Parallelizing Multi-head Attention

- The output of each of the h heads is of shape $N$ x $d_v$
  ➔ output of the multi-head layer with h heads consists of h matrices of shape $N$ x $d_v$.
- To make use of these matrices in further processing, they are concatenated to produce a single output with dimensionality $N$ x $hd_v$.

- Finally, another linear projection $W^o \in R^{hd_v \text{ x } d}$ is used, that reshapes it to the original output dimension for each token.
- Multiplying the concatenated N x h $d_v$ matrix output by $W^o \in R^{hd_v \text{ x } d}$ yields the self-attention output **A** of shape [N d].

# Parallelizing Multi-head Attention

$$\mathbf{O} = \text{LayerNorm}(\mathbf{X} + \text{MultiHeadAttention}(\mathbf{X}))$$
$$\mathbf{H} = \text{LayerNorm}(\mathbf{O} + \text{FFN}(\mathbf{O}))$$

- i.e.

$$\mathbf{T}^1 = \text{MultiHeadAttention}(\mathbf{X})$$
$$\mathbf{T}^2 = \mathbf{X} + \mathbf{T}^1$$
$$\mathbf{T}^3 = \text{LayerNorm}(\mathbf{T}^2)$$
$$\mathbf{T}^4 = \text{FFN}(\mathbf{T}^3)$$
$$\mathbf{T}^5 = \mathbf{T}^4 + \mathbf{T}^3$$
$$\mathbf{H} = \text{LayerNorm}(\mathbf{T}^5)$$

- The notation FFN($T^3$) means that the same FFN is applied in parallel to each of the N embedding vectors in the window.

# Input and output: Position embeddings and the Language Model Head

# Token and Position Embeddings

- Given N input tokens, the input matrix **X** (of shape $[N \times d]$) has an embedding for each word in the context.
- This embedding is created by adding two distinct embeddings for each input
  - token embedding
  - positional embedding

# Token Embeddings

- Embedding matrix E has shape [$|V| \times d$ ].
  - One row for each of the $|V|$ tokens in the vocabulary.
  - Each word is a row vector of $d$ dimensions
- Given: string "*Thanks for all the*"
  - *1.* Tokenize (e.g. with BPE) and convert into vocab indices

    w = [5,4000,10532,2224]
  - 2. Select the corresponding rows from E, each row an embedding

    (row 5, row 4000, row 10532, row 2224).
  - Alternatively, use 1-hot encoding, multiplying E by the 1-hot vectors.

# Position Embeddings

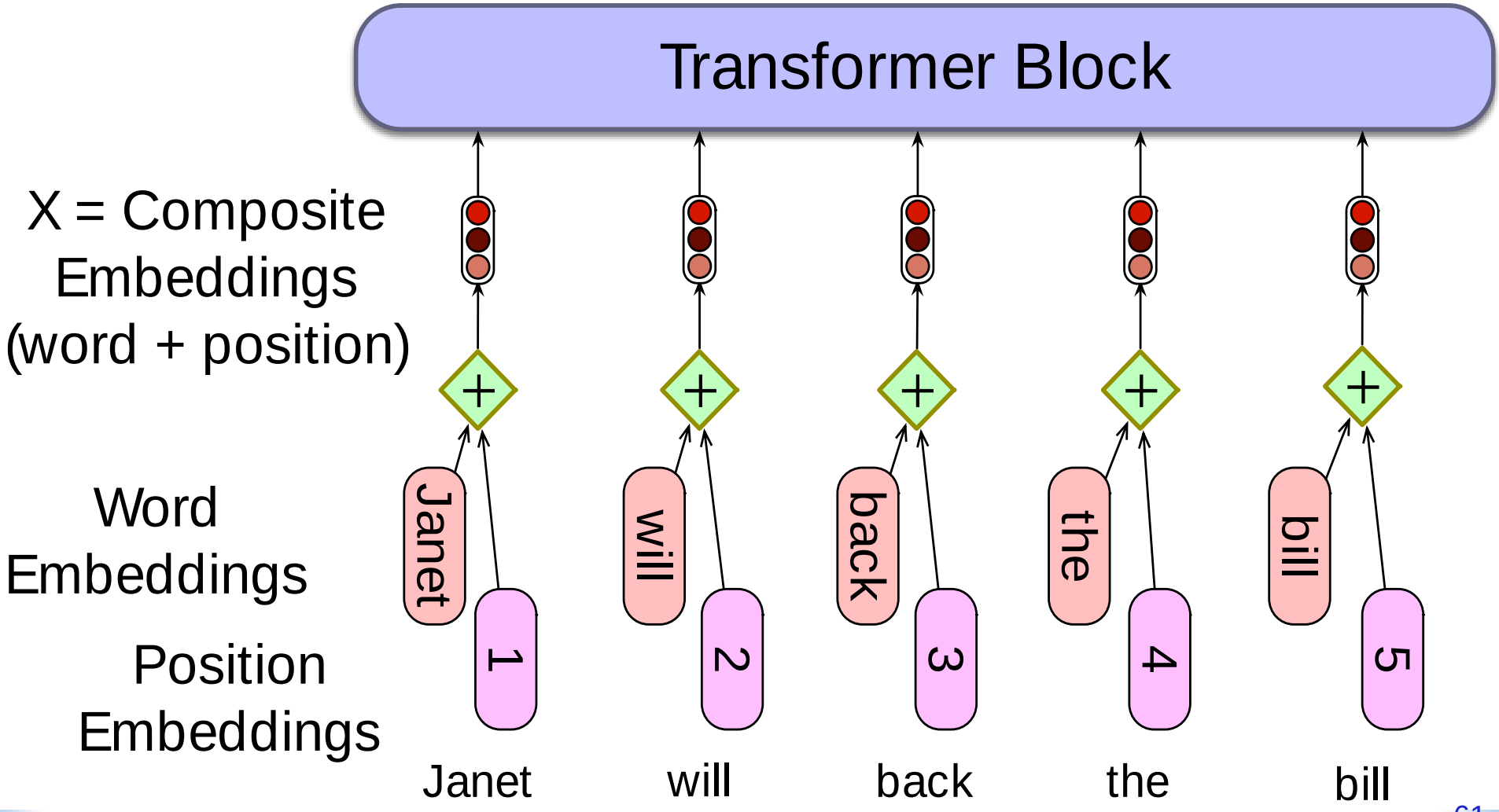
- There are many methods, but we'll just describe the simplest: absolute position.
- Goal: learn a position embedding matrix $\boldsymbol{E}\mathbf{pos}$ of shape $[1 \times N]$.
- Start with randomly initialized embeddings, one for each integer up to some maximum length.

(i.e., just as we have an embedding for token *fish*, we'll have an embedding for position 3 and position 17).

- As with word embeddings, these position embeddings are learned along with other parameters during training.

- To produce an input embedding that captures positional information, add the word embedding for each input to its corresponding positional embedding.

# Composite embeddings

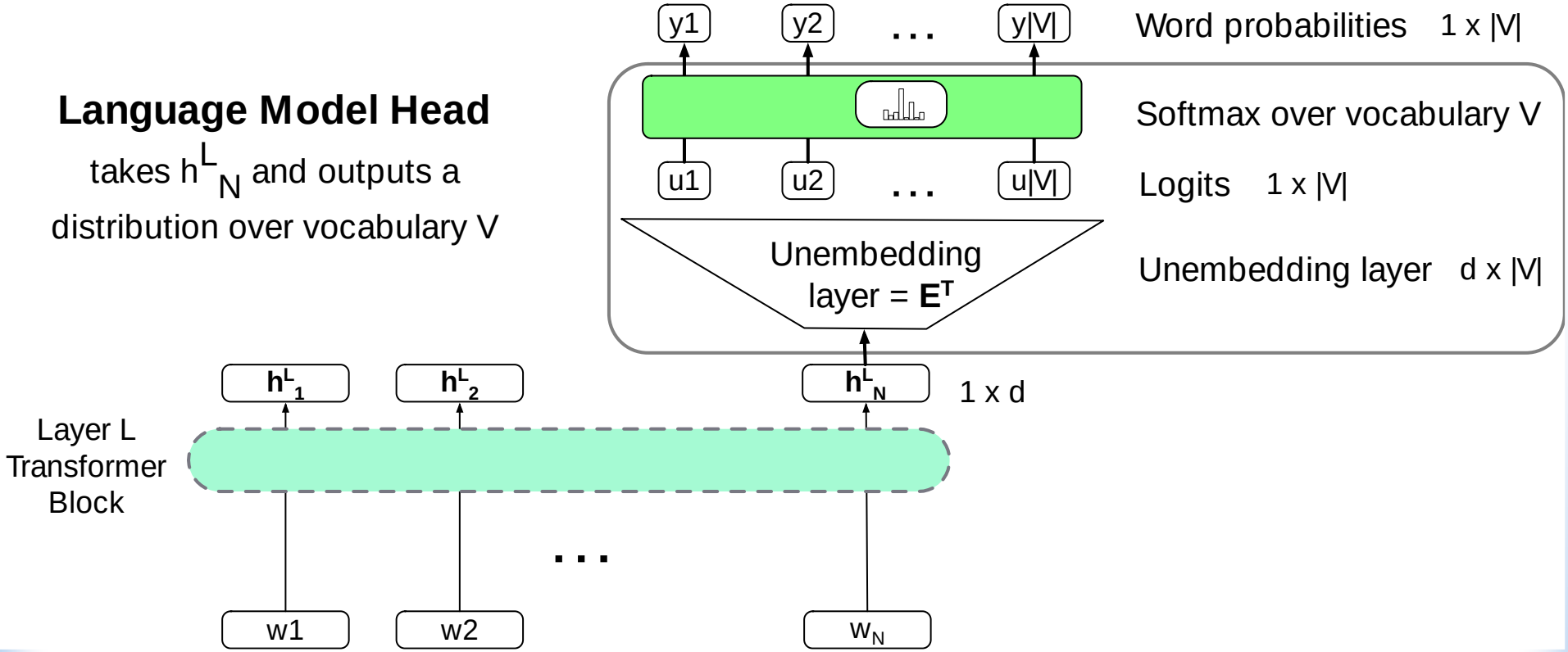


# Alternatives to Position Embeddings

- Potential problem with the *simple absolute position embedding approach:*
  - there will be plenty of training examples for the initial positions in our inputs and
  - correspondingly fewer at the outer length limits.
  - ➔ These latter embeddings may be poorly trained and may not generalize well during testing.
- Alternative approach to absolute positional embeddings:
  - Choose a static function that maps integer inputs to real-valued vectors in a way that captures the inherent relationships among the positions.
  - That is, it captures the fact that position 4 in an input is more closely related to position 5 than it is to position 17.
  - A combination of sine and cosine functions with differing frequencies was used in the original transformer work.
- Even more complex positional embedding methods exist, such as ones that represent relative position instead of absolute position.

# Language modeling head

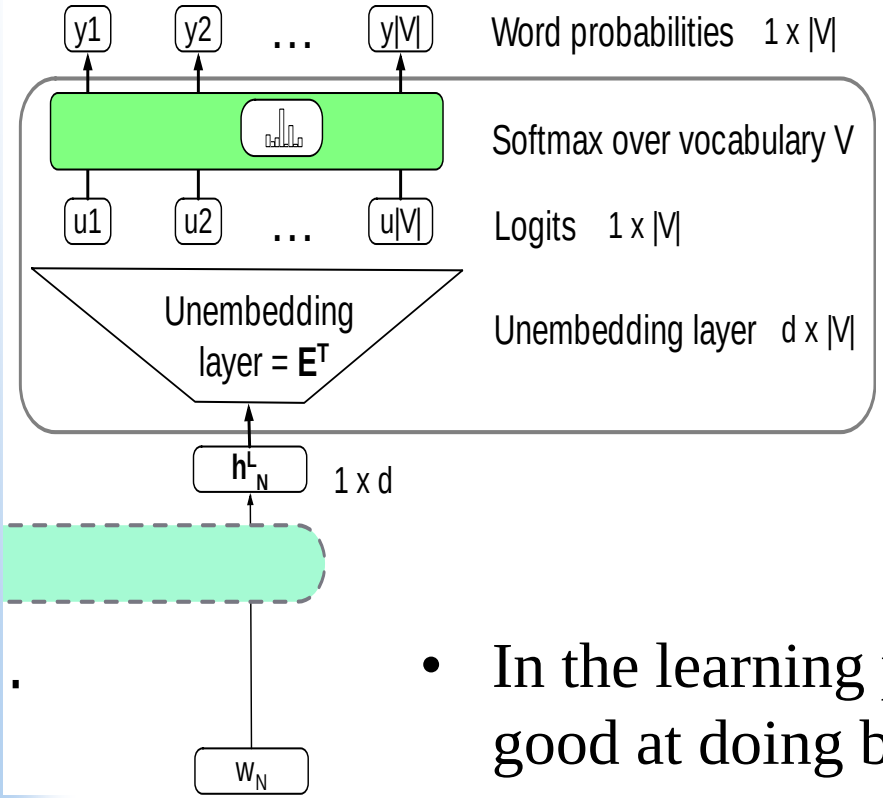
- Last component of the transformer: Given a context of words, they assign a probability to each possible next word.




- For transformer language models, the context is the size of the transformer's context window, which can be quite large: 2K, 4K, even 32K tokens for very large models.

# Language modeling head

- Job of language modeling head: take output of the final transformer layer from the last token N and use it to predict the upcoming word at position N+1.
- **Unembedding layer**: linear layer projects from $h^L_N$ (shape $[1 \times d]$) to logit vector (score vector).

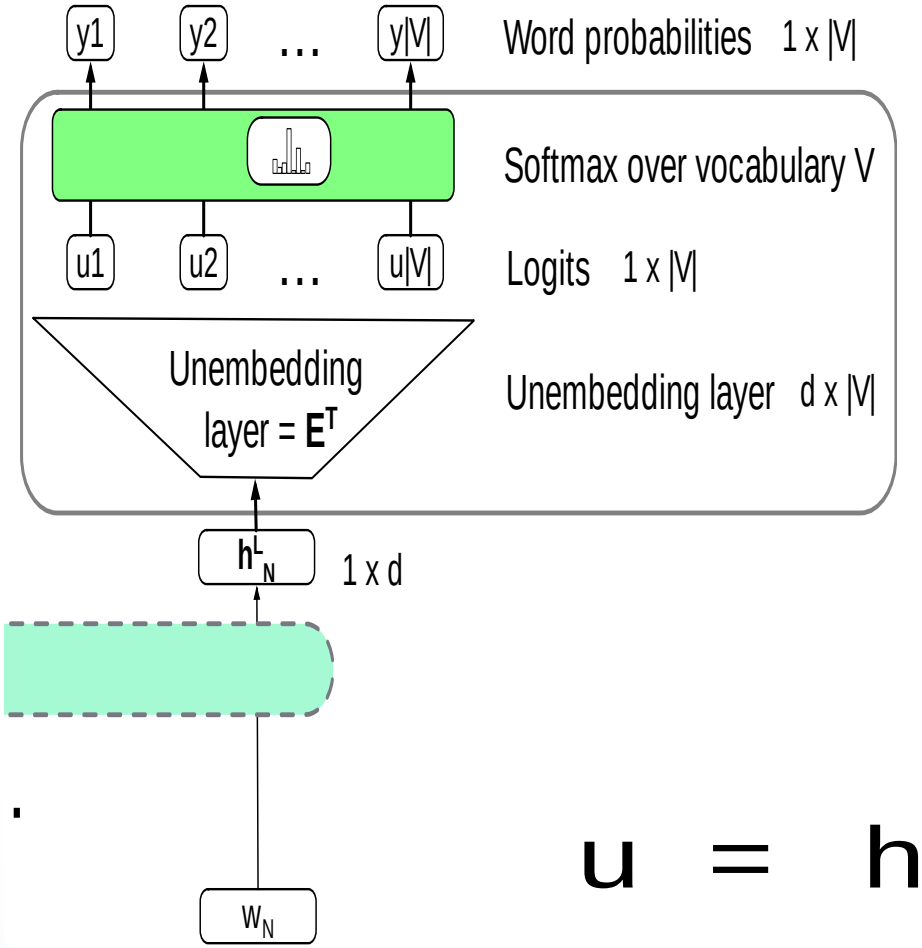



- Unembedding layer: **Tied** to $\mathbf{E}^{\mathbf{T}}$
- **Weight tying**, we use the same weights for two different matrices

- Unembedding layer maps from an embedding (shape $[1 \times d]$) to a 1x|V| vector over the vocabulary

- In the learning process, E will be optimized to be good at doing both of these mappings.

# Language modeling head




**Logits**, the score vector u

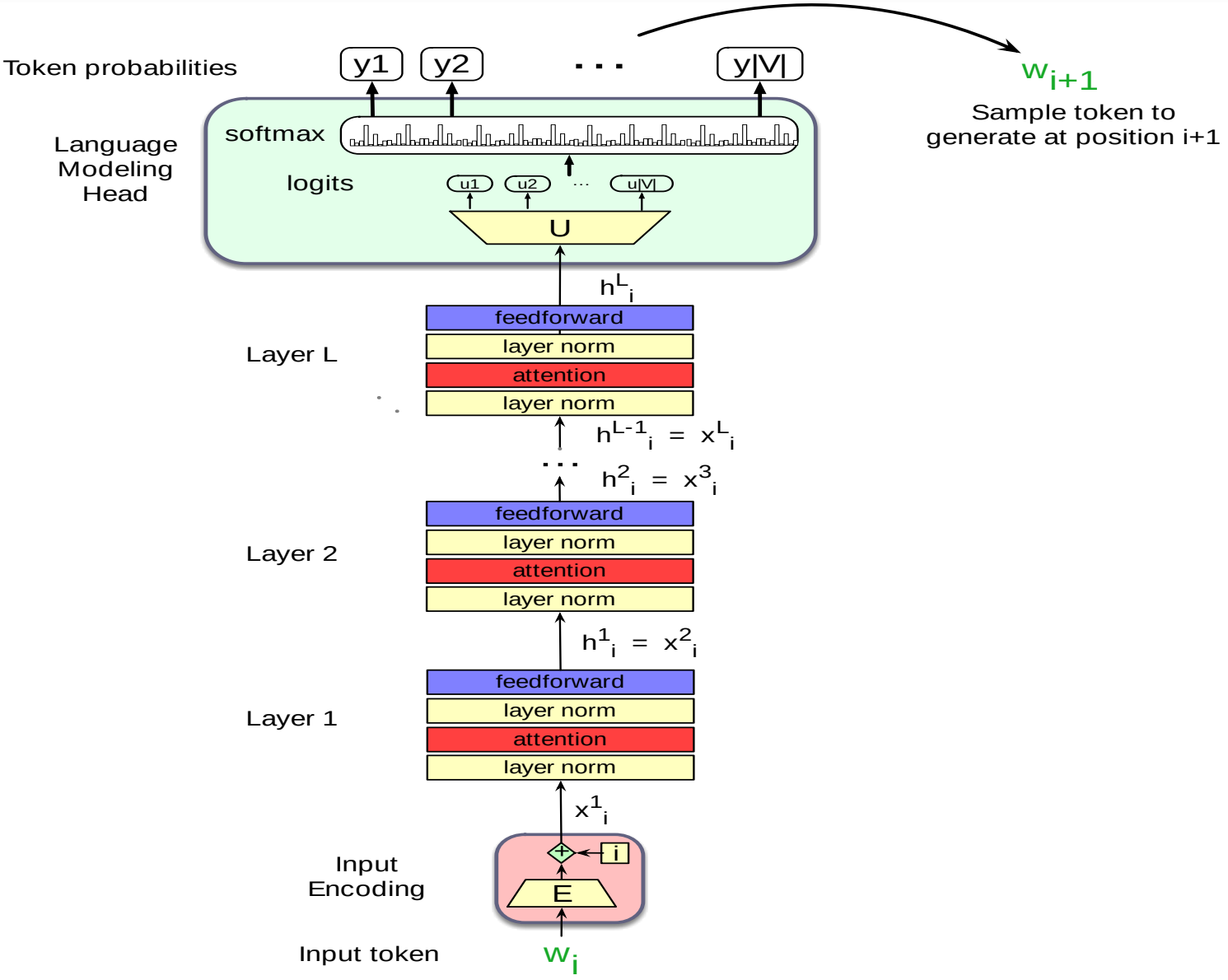
One score for each of the $|V|$ possible words in the vocabulary $V$. (Shape $1 \times |V|$.)

**Softmax** turns the logits into probabilities over vocabulary. Shape $1 \times |V|$.

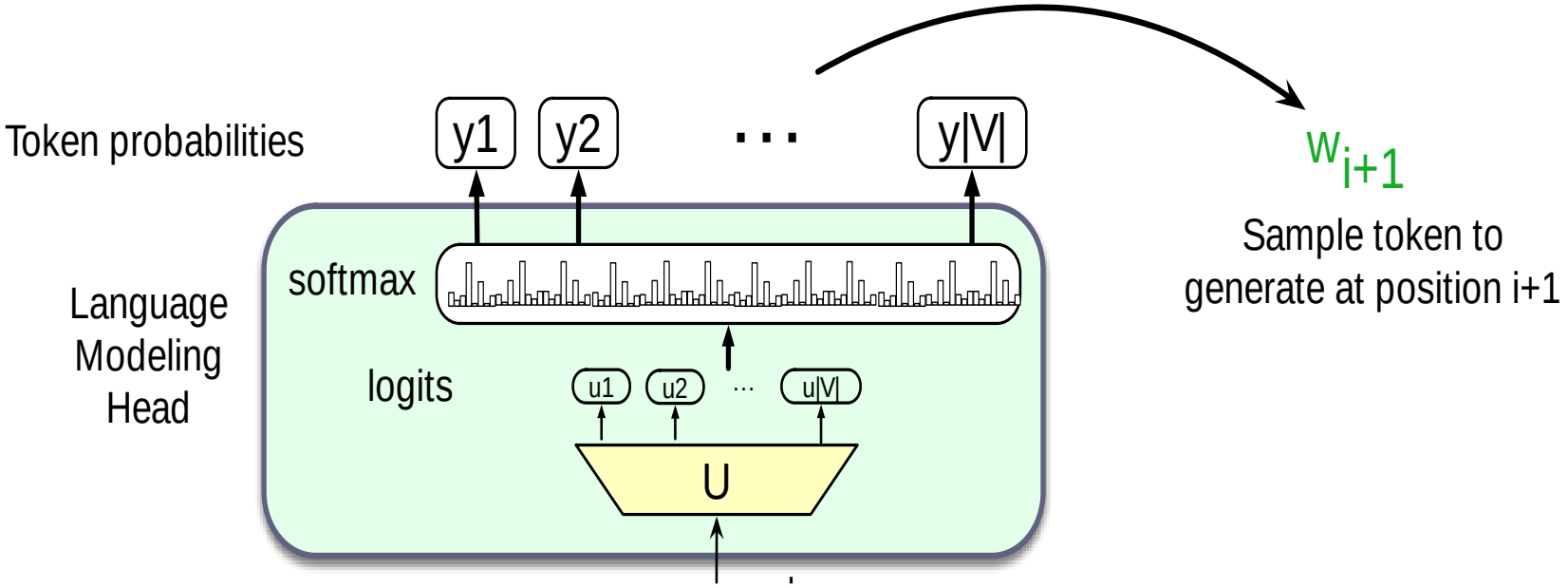
$$u = h_N^L E^T$$

$$y = \text{softmax}(u)$$

# The final transformer model



# The final transformer model



# Learning of the Multiple Heads

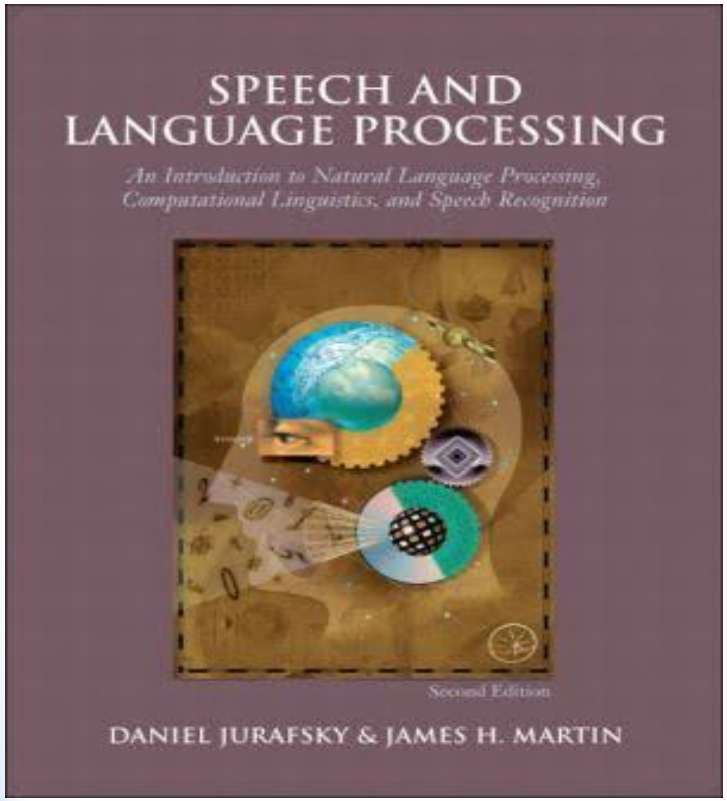
- After all heads compute their attention results, these are concatenated and passed through a linear transformation to produce the final output.
- The concatenation of the different outputs from all the heads allows the model to use the diverse features that each head has learned.
- ➔ even if each head individually doesn't produce the same result, the final output benefits from the combination of the distinct insights provided by each head.

# Missed point

- **How is $d_k$ determined?**
- In the context of the **scaled dot-product attention** used in Transformers, $\mathbf{d_k}$ is a hyperparameter chosen before training and is typically fixed.
- Usually chosen to be a smaller value than d, the input embedding dimension.
  - For example, if the embedding size d is 512, a common choice for $\mathbf{d_k}$ might be 64 or 128.
- The value of $\mathbf{d_k}$ controls how much attention each token can "focus on" relative to other tokens in the sequence.
- $\mathbf{d_k}$ also affects the complexity of the attention mechanism, as larger values of $\mathbf{d_k}$ result in larger matrices and more computation during attention.

# Textbook

Jurafsky, D. and Martin, J.H. (2024) *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models* (3rd ed. draft), Prentice Hall.
https://web.stanford.edu/~jurafsky/slp3/ed3bookaug20_2024.pdf




The draft version of the 3rd edition will be used; it has been updated to include more recent topics such as Vector Semantics and Embeddings, ANNs and Deep Learning for NLP, Transformers, etc.

This course **slides** are largely based on this textbook and its authors' slides with light customisations whenever seen fit.